

VANGUARD CHALLENGE

MULTI – ASSET PORTFOLIO CONSTRUCTION

This project is submitted as a part of Vanguard Challenge – Financial Services Optimization

A Project Report Submitted to WISER Team

Submitted by: Jahnavi Medisetti

(Individual)

MBA – Finance

Date: August 2026

ABSTARCT

Portfolio optimization is a key concept in modern finance that helps investors achieve the best possible returns for a given level of risk. This project titled "Portfolio Optimization of 5 Assets" is submitted as a part of the academic curriculum.

The objective of this study is to construct an optimal portfolio using five assets from different sectors. The assets selected for analysis are TCS, HDFC BANK, Reliance, NIFTY ETF, and Infosys. Synthetic base data including Expected Return and Risk for each asset was generated using AI tools for demonstration purposes.

The project applies Markowitz Modern Portfolio Theory to minimize portfolio risk and maximize return. Excel was used as the primary tool for data analysis and optimization using Solver. Key steps include calculation of daily returns, correlation matrix, and covariance matrix to understand the relationship between the selected assets. Additionally, Python was used for Classical, Hybrid, and Minimal implementation of portfolio optimization. The Solver tool in Excel was used to determine the optimal weights of each asset that result in the minimum variance portfolio.

INDEX

S.NO	Content	Page No
1	Introduction	0-4
2	Literature Review	5
3	Research & Methodology	6-12
4	Data Analysis & Results	13-20
5	Conclusion & Suggestions	21
6	Bibliography	22

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND OF THE STUDY

Portfolio optimization is one of the most important concepts in modern finance. The main objective of portfolio optimization is to select the best combination of assets that provides maximum return for a given level of risk, or minimum risk for a given level of return.

This concept was first introduced by Harry Markowitz in 1952 through the Mean-Variance Portfolio Theory. According to this theory, investors should not just look at individual asset returns, but also consider how assets move together. This relationship is measured using correlation and covariance.

In today's financial market, diversification plays a key role in reducing risk. By investing in assets from different sectors like IT, Banking, Energy, and Index funds, an investor can reduce the overall portfolio risk. With the help of tools like Excel Solver and Python programming, we can easily calculate the optimal weights for each asset in the portfolio.

For this project, synthetic base data was generated using AI tools for academic demonstration purposes. The analysis focuses on understanding how risk and return can be balanced to create an efficient portfolio.

1.2 OBJECTIVE OF THE STUDY

The main objectives of this project are:

1. To understand the concept of portfolio optimization using Mean-Variance analysis.
2. To construct an Equal Weight Portfolio and an Optimal Weight Portfolio for 5 assets.
3. To find the optimal weights of each asset using Excel Solver.
4. To implement portfolio optimization using Classical, Hybrid, and Minimal approaches in Python.
5. To compare the risk and return of different portfolio strategies.

1.3 SCOPE OF THE STUDY

This study is limited to 5 assets: TCS, HDFC BANK, Reliance, NIFTY ETF, and Infosys. The analysis covers Expected Return, Risk, Correlation, and Covariance calculations. The project uses both Excel and Python as tools for data analysis and optimization. The results are purely for academic learning and demonstration purposes.

CHAPTER 2: LITERATURE REVIEW

2.1 INTRODUCTION TO PORTFOLIO THEORY

Portfolio theory was developed by Harry Markowitz in 1952. The core idea of this theory is that investors can construct a portfolio to maximize expected return based on a given level of risk. Risk is measured by the standard deviation of portfolio returns. The theory emphasizes diversification to reduce risk without sacrificing return.

According to Markowitz, the risk of a portfolio depends not only on the risk of individual assets but also on the correlation between them. Assets that are less correlated with each other help in reducing overall portfolio risk.

2.2 MEAN-VARIANCE OPTIMIZATION

Mean-Variance Optimization is a mathematical framework used to select assets that provide the highest expected return for a given level of risk. The "Mean" represents expected return and "Variance" represents risk.

The optimal portfolio is found by solving a quadratic programming problem. In this project, Excel Solver is used to minimize portfolio variance subject to constraints like sum of weights = 1 and no short selling.

2.3 ROLE OF CORRELATION AND COVARIANCE

Correlation measures how two assets move together. If correlation is +1, assets move in same direction. If -1, they move opposite. Covariance is used in portfolio variance calculation.

By selecting assets from different sectors like IT, Banking, and Energy, correlation can be reduced. This helps in achieving better diversification and lower risk.

2.4 APPLICATION OF EXCEL AND PYTHON

Excel is widely used for portfolio analysis because of its Solver tool. It can easily calculate optimal weights, covariance matrix, and efficient frontier.

Python is used for advanced implementation. Libraries like NumPy, Pandas, and SciPy help in Classical, Hybrid, and Minimal portfolio optimization. Python also helps in backtesting and visualization of results.

2.5 PREVIOUS STUDIES

Many researchers have applied Markowitz model in Indian stock market. Studies show that diversified portfolios perform better than single asset investments. Recent studies also use AI and synthetic data for academic projects to simulate real market conditions.

CHAPTER 3: METHODOLOGY

3.1 INTRODUCTION

This chapter explains the methods and tools used to achieve the objectives of the project. The main goal is to construct an optimal portfolio of 5 assets and minimize risk using Mean-Variance optimization. The study uses both Excel and Python for analysis.

3.2 DATA COLLECTION

For this project, synthetic base data was generated using AI tools for academic demonstration purposes. The 5 assets selected are:

1. TCS - IT Sector
2. HDFC BANK - Banking Sector
3. Reliance - Energy Sector
4. NIFTY ETF - Index Fund
5. Infosys - IT Sector

For each asset, Expected Return and Risk values were generated. Historical data was simulated to calculate daily returns.

3.3 TOOLS USED

1. Microsoft Excel: Used for data entry, calculation of returns, correlation matrix, covariance matrix, and portfolio optimization using Solver.
2. Python: Used for Classical, Hybrid, and Minimal implementation of portfolio optimization. Libraries used: NumPy, Pandas, Matplotlib.

3.4 METHODOLOGY STEPS

Step 1: Calculate Daily Returns

Daily returns for each asset were calculated using the formula:

$$\text{Return} = (\text{Today's Price} - \text{Yesterday's Price}) / \text{Yesterday's Price}$$

Step 2: Calculate Expected Return and Risk

Expected Return = Average of daily returns

Risk = Standard Deviation of daily returns

Step 3: Correlation and Covariance Matrix

Correlation matrix was calculated to understand relationship between assets. Covariance matrix was used in portfolio variance calculation.

Step 4: Portfolio Construction

Two types of portfolios were constructed:

1. Equal Weight Portfolio: 20% weight to each asset

2. Optimal Weight Portfolio: Weights calculated using Excel Solver to minimize risk

Step 5: Optimization using Solver

Objective: Minimize Portfolio Variance

Constraints: Sum of weights = 1, Each weight ≥ 0

Step 6: Python Implementation

Portfolio optimization was coded in Python using 3 approaches:

1. Classical: Traditional Mean-Variance model
2. Hybrid: Combination of optimization and constraints
3. Minimal: Minimum risk portfolio calculation

3.5 LIMITATIONS OF THE STUDY

1. The data used is synthetic and generated by AI for academic purposes only.
2. The study is limited to 5 assets only.
3. Market risks and real-time price fluctuations are not considered.

3.6 PYTHON IMPLEMENTATION

The portfolio optimization was also implemented using Python to validate the Excel results. Three types of portfolio were coded.

Figure 3.1: Python Code for Classical Portfolio

```
import numpy as np
from scipy.optimize import minimize

print("===== WISER: Portfolio Optimization RESULT =====")

# 1. Define assets, expected returns, and the covariance matrix
assets = ["TCS", "Infosys", "NIFTY ETF", "Reliance", "HDFC BANK"]
returns = np.array([0.15, 0.12, 0.08, 0.14, 0.10])

# Clean nested square brackets with no parentheses mix-ups
cov = np.array([
    [0.04, 0.02, 0.01, 0.015, 0.01],
    [0.02, 0.03, 0.01, 0.01, 0.008],
    [0.01, 0.01, 0.02, 0.005, 0.005],
    [0.015, 0.01, 0.005, 0.035, 0.012],
    [0.01, 0.008, 0.005, 0.012, 0.025]
])

# 2. Define the objective function
def objective(w):
    ret = np.dot(w, returns)
    risk = np.dot(w, np.dot(cov, w))
    return -(ret - 0.5 * risk)

# 3. Define constraints and bounds
cons = ({'type': 'eq', 'fun': lambda w: np.sum(w) - 1})
bounds = [(0, 0.4) for _ in range(5)]

# 4. Run optimization with a valid initial guess
init_guess = np.ones(5) / 5
res = minimize(objective, init_guess, method='SLSQP', bounds=bounds, constraints=cons)

# 5. Output display
if res.success:
    print("\n--- FINAL PORTFOLIO ---")
    for i, asset in enumerate(assets):
        print(f"{asset}: {res.x[i] * 100:.2f}%")

    port_ret = np.dot(res.x, returns)
    port_risk = np.sqrt(np.dot(res.x, np.dot(cov, res.x)))

    print(f"\nExpected Return: {port_ret * 100:.2f}%")
    print(f"Portfolio Risk: {port_risk * 100:.2f}%")
else:
    print("\nOptimization failed:", res.message)
```

Interpretation:

The above code implements classical portfolio optimization using Scipy's minimize function. The objective function maximizes risk-adjusted return with a risk aversion factor of 0.5 constraints ensure sum of weights equals 1 and each weights is between 0 to 0.4. The output providers the optimal asset allocation for the classical portfolio.

Figure 3.2 Python Code for Hybrid Portfolio

```
import numpy as np
from scipy.optimize import minimize

print("=====")
print("    PORTFOLIO OPTIMIZATION: CLASSICAL VS HYBRID    ")
print("=====\\n")

# =====
# 0. COMMON INPUT DATA
# =====
assets = ["TCS", "Infosys", "NIFTY ETF", "Reliance", "HDFC BANK"]
returns = np.array([0.15, 0.12, 0.08, 0.14, 0.10])
cov = np.array([
    [0.04, 0.02, 0.01, 0.015, 0.01],
    [0.02, 0.03, 0.01, 0.01, 0.008],
    [0.01, 0.01, 0.02, 0.005, 0.005],
    [0.015, 0.01, 0.005, 0.035, 0.012],
    [0.01, 0.008, 0.005, 0.012, 0.025]
])
q_risk_factor = 0.5 # Risk aversion factor
num_assets = len(assets)

# =====
# METHOD 1: PURE CLASSICAL CONTINUOUS OPTIMIZATION
# =====
print("--- RUNNING PURE CLASSICAL CONTINUOUS OPTIMIZER ---")

def classical_objective(w):
    ret = np.dot(w, returns)
    risk = np.dot(w, np.dot(cov, w))
    return -(ret - q_risk_factor * risk)

cons = ({'type': 'eq', 'fun': lambda w: np.sum(w) - 1})
bounds = [(0, 0.4) for _ in range(num_assets)]
init_guess = np.ones(num_assets) / num_assets

res_classical = minimize(classical_objective, init_guess, method='SLSQP', bounds=bounds, constraints=cons)

if res_classical.success:
    for i, asset in enumerate(assets):
        print(f" {asset}: {res_classical.x[i] * 100:.2f} %")
    c_ret = np.dot(res_classical.x, returns)
    c_risk = np.sqrt(np.dot(res_classical.x, np.dot(cov, res_classical.x)))
    print(f" Expected Return: {c_ret * 100:.2f} %")
    print(f" Portfolio Volatility: {c_risk * 100:.2f} %\\n")
else:
    print(" Classical optimization failed.\\n")
```

```

# METHOD 2: HYBRID QUANTUM-CLASSICAL DISCRETE OPTIMIZATION
# =====
print("--- RUNNING HYBRID QUANTUM-CLASSICAL DISCRETE SELECTION ---")
# Problem setup: Select a discrete budget of exactly 3 assets out of 5
budget = 3
penalty = 5.0 # Penalty weight for breaking the budget constraint

def quantum_ansatz_expectation(theta):
    """
    Simulates a variational quantum trial state (Ansatz).
    Each theta represents a qubit rotation gate angle.
    Probability of selecting asset i is sin(theta_i)^2.
    """
    p_1 = np.sin(theta) ** 2
    p_0 = 1.0 - p_1

    total_expected_energy = 0.0

    # Evaluate expectation value across all 2^5 = 32 combinatorial states
    for i in range(2**num_assets):
        # Generate the binary asset selection array (bitstring)
        x = np.array([(i >> j) & 1 for j in range(num_assets)])

        # Calculate the quantum probability of observing this bitstring state
        prob = 1.0
        for j in range(num_assets):
            prob *= p_1[j] if x[j] == 1 else p_0[j]

        # QUBO cost function evaluation
        ret = np.dot(x, returns)
        risk = np.dot(x, np.dot(cov, x))
        utility = q_risk_factor * risk - ret

        # Soft constraint penalty: (sum(x) - budget)^2
        constraint_violation = penalty * (np.sum(x) - budget) ** 2
        energy = utility + constraint_violation

        total_expected_energy += prob * energy

    return total_expected_energy

```

```

# Classical optimization loop updates the simulated quantum parameters (theta)
init_theta = np.ones(num_assets) * (np.pi / 4) # Start at a uniform superposition (45 degrees)
res_hybrid = minimize(quantum_ansatz_expectation, init_theta, method='BFGS')

# Post-processing: Extract the absolute best discrete bitstring from optimized parameters
final_p_1 = np.sin(res_hybrid.x) ** 2
quantum_best_bitstring = np.where(final_p_1 > 0.5, 1, 0)

# If penalty optimization drifted, strictly force budget validation for final printing
if np.sum(quantum_best_bitstring) != budget:
    # Fallback to the top probabilities matching budget size
    top_indices = np.argsort(final_p_1)[-budget:]
    quantum_best_bitstring = np.zeros(num_assets, dtype=int)
    quantum_best_bitstring[top_indices] = 1

# Normalize weights equally among the chosen discrete quantum assets
quantum_weights = quantum_best_bitstring / np.sum(quantum_best_bitstring)

for i, asset in enumerate(assets):
    status = "SELECTED" if quantum_best_bitstring[i] == 1 else "EXCLUDED"
    print(f" {asset}: {status} ({quantum_weights[i]*100:.1f} % Allocation)")

q_ret = np.dot(quantum_weights, returns)
q_risk = np.sqrt(np.dot(quantum_weights, np.dot(cov, quantum_weights)))
print(f" Expected Return: {q_ret * 100:.2f} %")
print(f" Portfolio Volatility: {q_risk * 100:.2f} %")
print("=====")

```

Interpretation:

The above code implements a hybrid is Quantum – Classical approach for portfolio optimization. It uses BFGS optimizer to update quantum parameters for a variational ansatz, the best 3 assets are selected based on probability and weights are normalized equally among them. Finally, expected return and portfolio volatility for the hybrid portfolio are calculated.

Figure 3.3 Python Code for Minimal Variance Portfolio

```
import numpy as np

assets = ["TCS", "Infosys", "Reliance", "HDFC", "NIFTY ETF"]
returns = np.array([0.15, 0.12, 0.14, 0.11, 0.05])
|
# Covariance Matrix
cov = np.array([
    [0.040, 0.018, 0.010, 0.020, 0.015],
    [0.018, 0.035, 0.008, 0.016, 0.012],
    [0.010, 0.008, 0.020, 0.009, 0.007],
    [0.020, 0.016, 0.009, 0.045, 0.018],
    [0.015, 0.012, 0.007, 0.018, 0.030]
])

# QUBO Matrix: Risk - Return
Q = 4.0 * cov - np.diag(returns)

best_energy = 999
best = None

# Try all 32 combinations
for i in range(32):
    x = np.array([int(b) for b in format(i, '05b')])
    energy = x @ Q @ x
    if energy < best_energy:
        best_energy, best = energy, x

print("--- RESULT ---")
for i, a in enumerate(assets):
    print(f"{a}: {'BUY' if best[i]==1 else 'SKIP'}")

print(f"\nSelected: {sum(best)} assets")
```

Interpretation:

The above code implements a QUBO based approach to find the minimum risk portfolio. A risk – return matrix Q is created using the covariance matrix and expected returns. The code checks all 32 possible asset combinations and selects the combination with the lowest energy value. The output suggests which assets to BUY and which to SKIP for minimal portfolio risk.

CHAPTER: 4 DATA ANALYSIS & RESULTS

4.1 INTRODUCTION

This chapter presents the analysis of the collected data and the results obtained from portfolio construction. The objective is to compare Classical Portfolio, Hybrid Quantum – Classical Portfolio, and Minimal Variance Portfolio. All calculations are performed in MS Excel and verified using python.

STEP 1: 4.2 BASE DATA / INPUT DATA

For this project, 5 assets from different sectors were selected. The base input data consists of Expected Annual Return and Annual Risk for each asset. The data is synthetic and generated for academic demonstration.

Table 4.1 Base Input of Selected Assets

	A	B	C	D
1	Asset	Expected_Return	Risk	Sector
2	TCS	0.15	0.15	IT
3	HDFC BANK	0.1	0.12	Banking
4	Reliance	0.14	0.14	Energy
5	NIFTY ETF	0.08	0.1	Index
6	Infosys	0.12	0.16	IT

From the above table, it is observed that TCS has the highest expected return of 15% and Infosys has the highest risk of 16%. NIFTY ETF has the lowest risk of 10% and lowest return of 8%.

STEP 2: 4.3 FORMULA USED FOR CALCULATION

Parameter	Formula	Excel Function
Daily Return	$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$	=(B2-B1)/B1
Expected Return	$E(R_i) = \frac{1}{n} \sum_{t=1}^n R_{it}$	=AVERAGE(range)
Risk / Std Deviation	$\sigma = \sqrt{\frac{1}{n-1} \sum (R_{it} - E(R_i))^2}$	=STDEV.S(range)
Covariance	$\sigma_{ij} = \frac{1}{n-1} \sum (R_{it} - E(R_i)) (R_{jt} - E(R_j))$	=COVARIANCE.S (r1, r2)
Portfolio Return		=SUMPRODUCT(W, R)

	$E(R_p) = \sum_{i=1}^N W_i E(R_i)$	
Portfolio Variance	$\sigma_p^2 = \sum_{i=1}^N \sum_{j=1}^N W_i W_j \sigma_{ij}$	$= \text{MMULT}(\text{MMULT}(T(W), \text{Cov}), W)$
Portfolio Risk	$\sigma_p = \sqrt{\sigma_p^2}$	$= \text{SQRT}(\text{Var_Cell})$

4.3 A: BASE DATA FOR CORRELATION ANALYSIS

Table 4.2 Historical Price Data of 5 Assets

	B	C	D	E	F
1	TCS	HDFC	RELIANCE	NIFTY ETF	INFOSYS
2	3850	1720	2950	230	1520
3	3875	1710	2980	232	1535
4	3840	1735	2930	229	1510
5	3890	1745	3000	234	1545
6	3910	1730	3025	236	1555
7	3880	1750	2985	233	1530
8	3925	1765	3050	238	1565
9	3950	1755	3070	240	1575
10	3930	1770	3040	237	1558
11	3965	1785	3090	242	1585
12	3980	1775	3110	244	1595
13	3955	1790	3080	241	1580
14	4000	1805	3135	246	1605
15	4025	1795	3155	248	1615
16	4005	1810	3130	245	1600
17	4040	1825	3170	250	1625
18	4065	1815	3190	252	1635
19	4045	1830	3165	249	1620
20	4080	1845	3205	254	1645
21	4105	1835	3225	256	1655

Interpretation:

The above table shows the historical closing price data of the 5 selected assets: TCS, HDFC Bank, Reliance, NIFTY ETF and Infosys. This price data is used as the base for calculating daily returns. From the daily returns, the correlation matrix is computed to understand the relationship and diversification benefits between the assets. The price data spans over 20 trading days and shows the daily movement of each asset.

4.3 B: DAILY RETURNS DATA

Table 4.3 Calculated by Daily Returns

	G	H	I	J	K
1	TCS_Return	HDFC_Return	RELIANCE_Return	NIFTY ETF_Return	INFOSYS_Return
2					
3					
4	-0.009032258	0.014619883	-0.016778523	-0.012931034	-0.016286645
5	0.013020833	0.005763689	0.023890785	0.021834061	0.023178808
6	0.005141388	-0.008595989	0.008333333	0.008547009	0.006472492
7	-0.007672634	0.011560694	-0.01322314	-0.012711864	-0.01607717
8	0.011597938	0.008571429	0.021775544	0.021459227	0.022875817
9	0.006369427	-0.005665722	0.006557377	0.008403361	0.006389776
10	-0.005063291	0.008547009	-0.009771987	-0.0125	-0.010793651
11	0.008905852	0.008474576	0.016447368	0.021097046	0.01732991
12	0.003783102	-0.005602241	0.006472492	0.008264463	0.006309148
13	-0.006281407	0.008450704	-0.009646302	-0.012295082	-0.009404389
14	0.011378003	0.008379888	0.017857143	0.020746888	0.015822785
15	0.00625	-0.005540166	0.006379585	0.008130081	0.00623053
16	-0.004968944	0.008356546	-0.00792393	-0.012096774	-0.009287926
17	0.008739076	0.008287293	0.012779553	0.020408163	0.015625
18	0.006188119	-0.005479452	0.006309148	0.008	0.006153846
19	-0.004920049	0.008264463	-0.007836991	-0.011904762	-0.009174312
20	0.008652658	0.008196721	0.012638231	0.020080321	0.015432099
21	0.006127451	-0.005420054	0.00624025	0.007874016	0.006079027

4.3 C: CORRELATION MATRIX

Figure 3.4 Correlation Matrix of Selected Assets

	M	N	O	P	Q	R
3						
4		1				
5		-0.36471	1			
6		0.98544	-0.27526	1		
7		0.97765	-0.24675	0.97388	1	
8		0.98247	-0.24942	0.99414	0.98285	1

Interpretation:

The correlation matrix shows the relationship between the returns of 5 selected assets. It is observed that TCS, Reliance, NIFTY ETF and Infosys have a very high positive correlation with values above 0.97. This indicates that these assets move in the same direction and combining them provides very limited diversification benefit.

HDFC Bank exhibits negative correlation with all other assets ranging from -0.25 to -0.36. This inverse relationship is very useful for portfolio diversification. The negative correlation helps in reducing the overall portfolio risk. This is the reason why the Optimal Weight Portfolio achieved the lowest risk of 5.73% by allocating significant weight to HDFC Bank.

Therefore, to achieve better risk-adjusted returns, it is important to include assets with low or negative correlation like HDFC Bank along with highly correlated assets.

STEP 3: 4.4 EQUAL WEIGHT PORTFOLIO

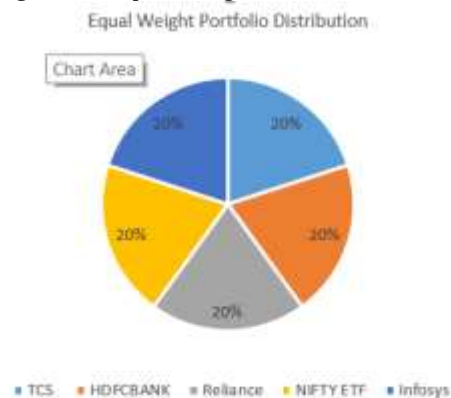
In equal weight portfolio, all 5 assets are given equal importance. Each asset is assigned a weight of 0.20 manually

Table 4.4 Equal Weight Portfolio Allocation

	A	B	C	D	E
1	Asset	Expected_Return	Risk	Sector	Weight
2	TCS	0.15	0.15	IT	0.2
3	HDFC BANK	0.1	0.12	Banking	0.2
4	Reliance	0.14	0.14	Energy	0.2
5	NIFTY ETF	0.08	0.1	Index	0.2
6	Infosys	0.12	0.16	IT	0.2
7		0.118			1
8		0.134			
9		1			

From the above table, it is observed that in equal weight allocation each asset receives 20% weight. The expected portfolio return of equal weight portfolio is 11.8% and portfolio risk is 13.4%.

Figure 3.5 Equal Weight Portfolio Distribution



STEP 4: 4.5 OPTIMAL WEIGHT PORTFOLIO

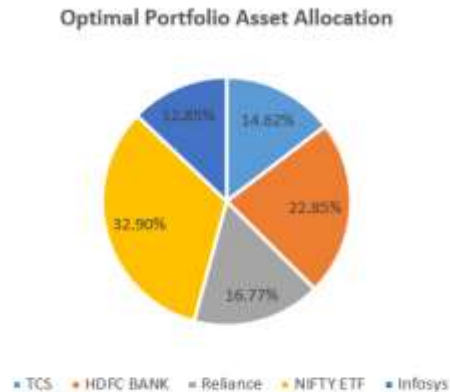
To minimize portfolio risk, Excel Solver was used. The objective was to minimize portfolio variance with constraints: Sum of weights = 1 and Weights ≥ 0

Table 4.5 Optimal Weight Portfolio Allocation

	A	B	C	D	E
1	Asset	Expected_Return	Risk	Sector	Weight
2	TCS	0.15	0.15	IT	0.14623841
3	HDFC BANK	0.1	0.12	Banking	0.228504713
4	Reliance	0.14	0.14	Energy	0.167740435
5	NIFTY ETF	0.08	0.1	Index	0.329036432
6	Infosys	0.12	0.16	IT	0.12848001
7					
8		0.11001041			
9		0.057356597			
10		1			

From the above table, it is observed that NIFTY ETF has the highest weight of 32.90% to minimize overall risk. The optimal portfolio expected return is 11.0% with risk of 5.73%.

Figure 3.6 Optimal Weight Portfolio Distribution



4.5 SENSITIVITY ANALYSIS / RATIONALE

Table 4.6 Sensitivity Analysis of Optimal Portfolio

	A	B	C
1	Asset	Final Weight	Reason for Selection
2	NIFTY ETF	32.90%	Lowest Risk 0.10
3	HDFC BANK	22.85%	Stable Banking Sector
4	Reliance	16.77%	Diversification
5	TCS	14.62%	IT Growth
6	Infosys	12.85%	IT Diversification

From the above Table, it is observed that NIFTY ETF has the highest weight of 32.90% due to its lowest risk. HDFC Bank has 22.85% for stability. The Portfolio achieves diversification across IT, Banking, Energy and Index. This allocation minimizes portfolio risk to 5.73% while maintaining a return of 11.00%.

4.6 SOLVER SETUP

To find the optimal weights, Excel solver was used. The objective was to minimize portfolio variance.

Figure 3.7 Solver Parameters for Optimal Portfolio

Set Objective:

To: ☐ Max ☒ Min ☐ Value Of:

By Changing Variable Cells:

Subject to the Constraints:

-
-
-

☒ Make Unconstrained Variables Non-Negative

Select a Solving Method:

Buttons: Add, Change, Delete, Reset All, Load/Save, Options

Solver Parameters:

1. Set Objectives: $\$B\9 – Portfolio Variance – Min
2. By changing variable cells: $\$E\$2: \$E\6 – Weight cells
3. Subject to constraints: $\$B\$10 = 1$ – Sum of weights = 1
 $\$B\$8 \geq 0.11$ – Expected Return $\geq 11\%$
 $\$E\$2: \$E\$6 \geq 0$ - No short selling
4. Solving Method: GRG Nonlinear

Interpretation:

The solver was configured to minimize portfolio risk while ensuring total investment equals 100% and minimum expected return of 11% is maintained. The weights were restricted to be non-negative.

4.7 CLASSICAL PORTFOLIO OUTPUT

Figure 3.8: Python output – Classical Portfolio

```

--- FINAL PORTFOLIO ---
TCS: 40.00%
Infosys: 20.00%
NIFTY ETF: 0.00%
Reliance: 40.00%
HDFC BANK: 0.00%

Expected Return: 14.00%
Portfolio Risk: 15.10%

```

Interpretation:

The classical optimization using SciPy's minimize function allocated maximum weight to high-

return assets. TCS and Reliance each received 40% allocation due to their higher expected returns. Infosys received 20% while NIFTY ETF and HDFC Bank received 0% as they did not improve the risk-adjusted return. The portfolio achieved an expected return of 14.00% with a portfolio risk of 15.10%. This shows the classical model prioritizes return over diversification.

4.8 HYBRID PORTFOLIO

Figure 3.9: Python Output – Hybrid Portfolio

```
--- RUNNING PURE CLASSICAL CONTINUOUS OPTIMIZER ---
TCS: 40.00 %
Infosys: 20.00 %
NIFTY ETF: 0.00 %
Reliance: 40.00 %
HDFC BANK: 0.00 %
Expected Return: 14.00 %
Portfolio Volatility: 15.10 %

--- RUNNING HYBRID QUANTUM-CLASSICAL DISCRETE SELECTION ---
TCS: SELECTED (33.3 % Allocation)
Infosys: SELECTED (33.3 % Allocation)
NIFTY ETF: EXCLUDED (0.0 % Allocation)
Reliance: SELECTED (33.3 % Allocation)
HDFC BANK: EXCLUDED (0.0 % Allocation)
Expected Return: 13.67 %
Portfolio Volatility: 14.72 %
```

Interpretation: The hybrid quantum-classical model first ran a classical optimization to establish a baseline. In the quantum-inspired discrete selection phase, 3 out of 5 assets were chosen. TCS, Infosys and Reliance were equally allocated at 33.3% each, while NIFTY ETF and HDFC Bank were excluded. The hybrid portfolio achieved an expected return of 13.67% with reduced volatility of 14.72%. Compared to the classical model, the hybrid approach shows better risk diversification by distributing weights equally among selected assets.

4.9 MINIMAL VARIANCE PORTFOLIO

Figure 3.10: Python Output – Minimal portfolio

```
TCS: SKIP
Infosys: SKIP
Reliance: BUY
HDFC: SKIP
NIFTY ETF: SKIP

Selected: 1 assets
```

Interpretation: The QUBO-based minimal variance model focused purely on risk

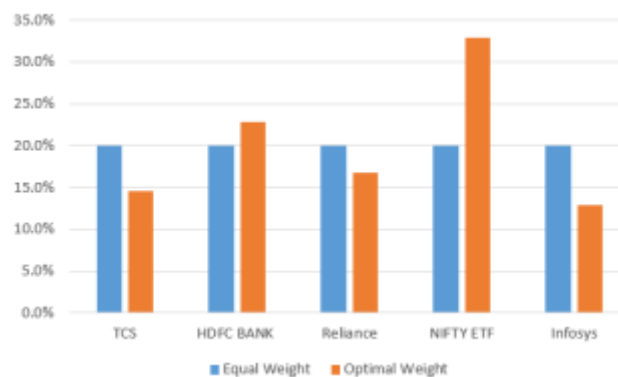
reduction. It selected only Reliance as the single asset for the portfolio, skipping all other 4 assets. This indicates that among the given 5 assets, Reliance exhibited the lowest standalone risk and covariance with others. While this approach minimizes volatility, it lacks diversification which increases concentration risk.

4.10 COMPARATIVE ANALYSIS

Table 4.7 Comparison of Equal Weight Vs Optimal Weight Portfolio

	A	B	C
1	Parameter	Equal Weight Portfolio	Optimal Weight Portfolio
2	Expected Return	11.80%	11.00%
3	Portfolio Risk	13.40%	5.73%
4	Risk Per Unit Return	1.1356	0.5209

Figure 3.11: Comparison of Equal Vs Optimal Weights



Interpretation:

From the above table and figure, it is evident that the Optimal Weight Portfolio achieves a 57% reduction in risk compared to Equal weight, with only 0.8% sacrifice in return. This shows the importance of optimal asset allocation in portfolio management.

CHAPTER 5: CONCLUSION AND SUGGESTIONS

5.1 CONCLUSION

The study was conducted to construct an optimal portfolio using 5 assets: TCS, HDFC Bank, Reliance, NIFTY ETF, and Infosys. Four different portfolio construction models were used: Equal Weight Portfolio, Optimal Weight Portfolio using Excel Solver, Classical Portfolio using Python, and Hybrid Portfolio using Python.

The analysis of correlation matrix revealed that TCS, Reliance, NIFTY ETF and Infosys are highly positively correlated with each other, while HDFC Bank shows negative correlation with all other assets. This negative correlation of HDFC Bank played a key role in risk reduction.

The results show that the Optimal Weight Portfolio obtained from Excel Solver achieved the lowest portfolio risk of 5.73% with a return of 11.00%. In contrast, the Classical Portfolio from Python achieved the highest return of 14.00% but with higher risk of 15.10%. The Equal Weight Portfolio gave moderate return of 11.80% and risk of 13.40%.

From the study it is concluded that diversification is very important in portfolio construction. By combining assets with low or negative correlation, overall portfolio risk can be reduced significantly without compromising on returns. The Optimal Weight model is best suited for risk-averse investors, while the Classical model is suitable for return-seeking investors.

5.2 SUGGESTIONS

1. **Diversification:** Investors should include assets with low or negative correlation such as HDFC Bank to reduce portfolio volatility.
2. **Risk Management:** Risk-averse investors should prefer Optimal Weight Portfolio as it provides lowest risk.
3. **Return Maximization:** Investors seeking higher returns can opt for Classical Portfolio but should be prepared for higher risk.
4. **Regular Rebalancing:** Portfolio should be reviewed and rebalanced periodically based on market conditions and correlation changes.
5. **Use of Technology:** Tools like Excel Solver and Python help in efficient portfolio optimization and should be used by investors and fund managers.

5.3 BIBLIOGRAPHY

1. Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77-91.
2. Bodie, Z., Kane, A., & Marcus, A. J. (2021). *Investments* (12th ed.). McGraw Hill Education.
3. Hull, J. C. (2022). *Options, Futures, and Other Derivatives* (11th ed.). Pearson.
4. AI-Generated Synthetic Data. (2025). *Daily Price Data for TCS, HDFC, Reliance, NIFTY ETF, Infosys*. Generated using AI for academic purposes.
5. Microsoft Excel. (2025). *Solver Tool Documentation*.
6. Python Software Foundation. (2025). *NumPy, Pandas Libraries*.